

Scenario 1: Online Food Delivery System

A food delivery application partners with multiple restaurants. Every restaurant prepares food differently, but all restaurants must support receiving an order and preparing it.

Design a system where:

- Every restaurant has a name and location.
- Every restaurant can display its details.
- The process of preparing food differs for each restaurant.
- Create restaurants such as **Pizza Restaurant**, **Biryani Restaurant**, and **Bakery**.

Task

- Create an appropriate abstract class.
- Include common variables and methods.
- Declare an abstract method for preparing food.
- Override the abstract method in each restaurant.
- Demonstrate runtime polymorphism using a parent reference.

Scenario 2: Ride Booking Platform

A cab booking application offers different transportation services.

Available services:

- Bike Ride
- Car Ride
- Auto Ride

Common features:

- Driver Name
- Vehicle Number
- Pickup Location
- Drop Location
- Display Ride Details

Different behavior:

- Fare calculation

Task

- Create an abstract class for all ride services.
- Define common properties and methods.
- Add an abstract method to calculate the fare.
- Implement the fare calculation differently for each ride type.
- Demonstrate runtime polymorphism.

3. Cloud Storage Platform

A software company provides cloud storage services through different platforms such as **Google Drive**, **Dropbox**, and **OneDrive**. Every storage service maintains common information like account name, storage capacity, and user details. Each service must allow users to upload files, download files, and display storage information. However, the algorithms used for synchronizing files, calculating available storage, and handling backups differ for each provider. The company wants a design that supports adding new cloud storage providers with minimal code changes.

Question:

Design the application using an **abstract class**. Create appropriate subclasses and implement provider-specific functionality while maintaining a common structure.

4. OTT Video Streaming Platform

A media company offers different subscription plans such as **Basic, Standard, and Premium**. Every subscription contains common information including subscriber ID, customer name, email address, and subscription start date. All plans allow users to log in, view their profile, and watch content. However, the video quality, number of supported devices, offline download limit, and monthly subscription cost vary depending on the selected plan. The company expects to launch additional subscription plans in the future.

Question:

Design the system using an **abstract class**. Identify the common features, define appropriate abstract methods, implement the subclasses, and demonstrate runtime polymorphism.